



LangChain



Resources

Why Use LangChain

For full detail docs: <https://reference.langchain.com/python/>

- Standardise workflow and provided commonly used helper functions (containing built-in advanced functionality such as streaming and durability)
- Presents great concepts & patterns to learn to use
- Low-level flexibility and can always remove LangChain bits and write them in own code if feel need
- LangChain also provides basic agent functions which are built on top of LangGraph



LangChain Agents

Models

LangChain uses “chat models” which is a class that can take in system, human, AI, tool messages

▼ Create Custom Chat Models

- Some providers use the same format as OpenAI and we can therefore use the OpenAI chat model and pass in new `base_url` and `api_key` arguments

https://reference.langchain.com/python/langchain_core/language_models/?_gl=1*atrcv6*_gcl_au*MTc4OTUyMzQ4OS4xNzYzODI3MTc5*_ga*MjEwMDU2Mjc0My4xNzYzODI3MTgw*_ga_47WX3HKKY2*czE3NjQ2ODM1ODUkbzcxJGcxJHQxNzY0Njg0MzMwJGo1NiRsMCRoMA..#langchain_core.language_models.BaseChatModel

Instantiate Model

- To initialise a chat model, `load_dotenv()` the API key and run the providers

`init_chat_model` :

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
    model="claude-sonnet-4-5-20250929",
    # Option Kwargs passed to the model:
    temperature=...,
    max_tokens=...,
    timeout=...,
    max_retries=...
)
```

- To get info on the model:

```
model.profile
{
  "max_input_tokens": 400000,
  "image_inputs": True,
  "reasoning_output": True,
  "tool_calling": True,
  ...
}
```

Call Model

Option 1: **Invoke**

- Calls model and returns response once fully generated

```
# Single user message call
```

```
response = model.invoke("Why do parrots have colorful feathers?")
```

```
# Multi-message call
```

```
conversation = [
```

```
    SystemMessage("You are a helpful assistant that translates English to French."),
```

```
    HumanMessage("Translate: I love programming."),
```

```
    AIMessage("J'adore la programmation."),
```

```
    HumanMessage("Translate: I love building applications.")
```

```
]
```

```
response = model.invoke(conversation)
```

```
# Equivalent to
```

```
conversation = [
```

```
    {"role": "system", "content": "You are a helpful assistant that translates English to French."},
```

```
    {"role": "user", "content": "Translate: I love programming."},
```

```
    {"role": "assistant", "content": "J'adore la programmation."},
```

```
    {"role": "user", "content": "Translate: I love building applications."}]
```

```
]
```

```
response = model.invoke(conversation)
```

Option 2: **Stream**

- Calls model and streams back tokens
- As opposed to `invoke()`, which returns a single `AIMessage` after the model has finished generating its full response, `stream()` returns multiple `AIMessageChunk` objects (tokens)

Option 1

```
for chunk in model.stream("Why do parrots have colorful feathers?"):
    print(chunk.text, end="|", flush=True)
```

The
sky
is
typically

Option 2

```
full = None # None | AIMessageChunk
for chunk in model.stream("What color is the sky?"):
    full = chunk if full is None else full + chunk
    print(full.text)
```

The
The sky
The sky is
The sky is typically

Option 3: **Batch**

- Send off multiple parallel calls (returns all at once and does not stream per token)

```
model.batch(  
    list_of_inputs,  
    # optional config  
    config={  
        'max_concurrency': 5,  
    }  
)  
for response in responses:  
    print(response)
```

Multi-Modality

- We handle this by using **content_blocks**

```
response = model.invoke("Create a picture of a cat")  
print(response.content_blocks)  
[  
    {"type": "text", "text": "Here's a picture of a cat"},  
    {"type": "image", "base64": "...", "mime_type": "image/jpeg"},  
]
```

Reasoning

- **If supported by the underlying model**, you can surface this reasoning process to better understand how the model arrived at its final answer.
- Refer to <https://docs.langchain.com/oss/python/integrations/providers/overview> for provider details

```
for chunk in model.stream("Why do parrots have colorful feathers?"):
    reasoning_steps = [r for r in chunk.content_blocks if r["type"] =
                       = "reasoning"]
    print(reasoning_steps if reasoning_steps else chunk.text)
```

Caching

- Some providers implement automatically, and some require you to specify where to cache
- See provider details at <https://docs.langchain.com/oss/python/integrations/providers/overview>

Logits

- Some models can return probabilities for tokens at each step

```
model = init_chat_model(  
    model="gpt-4o",  
    model_provider="openai"  
)  
model.bind(logprobs=True)  
  
response = model.invoke("Why do parrots talk?")  
print(response.response_metadata["logprobs"])
```


Messages

- Messages are the fundamental unit of context for models in LangChain. In almost all agents there will be a `messages` field in the state
- Messages are objects that contain:
 - **Message Type (Role):**
 - System Message: Tells the model how to behave and provide context for interactions
 - Human Message: Represents user input and interactions with the model
 - AI Message: Responses generated by the model, including text content, tool calls, and metadata
 - Tool Message: Represents the outputs of tool calls
 - **Content** - Represents the actual content of the message (like text, images, audio, documents, etc.)
 - **Metadata** - Optional fields such as response information, message IDs, and token usage

Basic Usage: Text String

```
response = model.invoke("Write a haiku about spring")
```

Main Usage: List of Messages

```
from langchain.messages import SystemMessage, HumanMessage,
AIMessage

messages = [
    SystemMessage("You are a poetry expert"),
    HumanMessage("Write a haiku about spring"),
    AIMessage("Cherry blossoms bloom...")
]
response = model.invoke(messages)
```

Other Usage

▼ OpenAI Chat Completion Style Messages Demo

```
messages = [
    {"role": "system", "content": "You are a poetry expert"},
    {"role": "user", "content": "Write a haiku about spring"},
    {"role": "assistant", "content": "Cherry blossoms bloom..."}
]
response = model.invoke(messages)
```

Multi-Modal Message

- Use `content_blocks` which do not enforce string typing to return images etc
- More info at <https://docs.langchain.com/oss/python/langchain/messages>

```
# List of standard content blocks
human_message = HumanMessage(content_blocks=[
    {"type": "text", "text": "Hello, how are you?"},
    {"type": "image", "url": "https://example.com/image.jpg"},
])
```

Reading Messages

- Messages are just a list with type, content and metadata labels which we can access with i.e. `.content`

```
# Invoke agent
result = agent.invoke({"messages": [{"role": "user", "content": "Tell me a joke"}]})
messages = result["messages"]

# Print all messages
print(messages)

# Get & print last message
last_msg = messages[-1]
print(last_msg)

# Get & print last 2 messages
last_two = messages[-2:]
print(last_two)
```

Memory

See:  Memory

Retrieval

LangGraph provides sets of helper functions for RAG... The standard unit they work with is a **LangGraph Document** data structure

Document Loaders

Provide a standard interface for reading data from different sources (such as Slack, Notion, or Google Drive) into LangChain's Document format

- See the types of document loaders at https://docs.langchain.com/oss/python/integrations/document_loaders
 - Includes web, PDF (i.e. docling), Cloud storage, social platform, common file types...

▼ Document Loader CSV Demo

- Lazy loading lets us process each document as it arrives (since cannot hold huge datasets into Python all at once)

```
from langchain_community.document_loaders.csv_loader import CSVLoader
```

```
loader = CSVLoader(  
    ... # Integration-specific parameters here  
)
```

```
# Load all documents  
documents = loader.load()
```

```
# For large datasets, lazily load documents  
for document in loader.lazy_load():  
    print(document)
```

Text Splitters

- For chunking
- Has many options - see all at <https://docs.langchain.com/oss/python/integrations/splitters>

Embedding Models

LangChain provides a standard interface for text embedding models (e.g., OpenAI, Cohere, Hugging Face)

- See all models at https://docs.langchain.com/oss/python/integrations/text_embedding

Vector Stores

- LangChain provides a unified interface for vector stores, allowing you to:
 - `add_documents` - Add documents to the store.
 - `delete` - Remove stored documents by ID.
 - `similarity_search` - Query for semantically similar documents.

▼ Vector Store Demo

- Note here documents are actually the chunks (as `LangChain Document` objects)

```
from langchain_core.vectorstores import InMemoryVectorStore
vector_store = InMemoryVectorStore(embedding=SomeEmbeddingModel())

# Add documents
vector_store.add_documents(documents=[doc1, doc2], ids=["id1", "id2"])

# Delete documents
vector_store.delete(ids=["id1"])

# Similarity search
similar_docs = vector_store.similarity_search("your query here")

# Similarity search with metadata filtering
vector_store.similarity_search(
    "query",
    k=3,
    filter={"source": "tweets"}
)
```

Retrievers

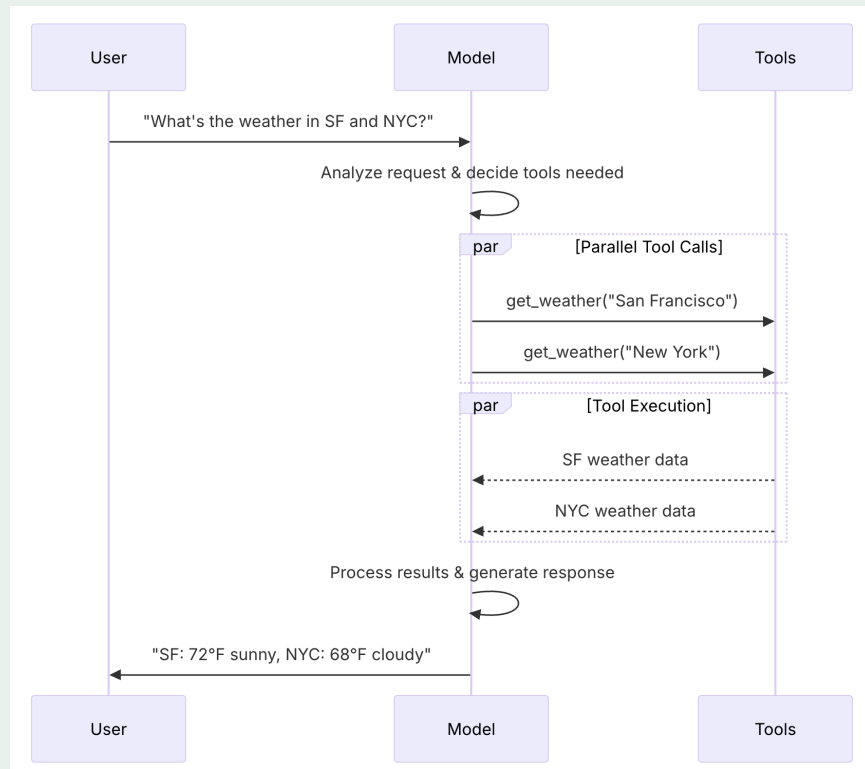
- Retrievers accept a string query as input and return a list of **Documents** as output.
This can work well with LangChain vector stores

Tools & Structured Output

Overview

Tools are pairings of:

- A (often JSON) schema, including the name of the tool, a description, and argument definitions
- A routine to execute.



When using a model separately from an agent, it is up to you to execute the requested tool and return the result back to the model for use in subsequent reasoning. When using an agent, the agent loop will handle the tool execution loop for you.

Tool Functions

- This should contain what the tool should do if called
- We should add `@tool` above any tool functions since this lets us provide custom details provided to the LLM about how to call the tool:

	Default (no <code>@tool</code>)	How to Customise What LLM Sees
Tool name	Uses the Python function name <code>multiply</code> → <code>"multiply"</code>	Set explicitly: <code>@tool(name="multiply_numbers")</code>
Tool description	Uses the function docstring (first line or full docstring)	Supply custom description via docstring OR override with <code>@tool(description="...")</code>
Tool Arguments	Names: Taken directly from tool function inputs Types: Taken directly from tool function input type hints Descriptions: Not provided	[Option 1] Customise Only Tool Argument Description: Use <code>@tool(parse_docstring=True)</code> and provide <code>Args:</code> section in docstring. [Option 2] Customise Tool Argument Names, Types & Descriptions: Define a Pydantic <code>args_schema</code> : <pre>class MultiplyArgs(BaseModel): a: int = Field(description="The left operand") b: int = Field(description="The right operand") @tool(name="multiply_custom", args_schema=MultiplyArgs) def multiply(args: MultiplyArgs) → int:</pre>

Define Tool Function

```
@tool(name="multiply_numbers", parse_docstring=True)
def multiply(args: MultiplyArgs) → int:
    """Multiply two integers.
```

Args:

a: The first integer.

b: The second integer.

"""

return a * b

Tool Functions: Advanced

▼ Providing State to Tool Definitions (Without Exposing it to LLM) Demo

- Do this by including `runtime: ToolRuntime`
- The LLM does not see this in the tool schema

```
@tool
def summarize_conversation(
    runtime: ToolRuntime
) → str:
    """Summarize the conversation so far."""
    messages = runtime.state["messages"]

    human_msgs = sum(1 for m in messages if m.__class__.__name__ == "HumanMessage")
    ai_msgs = sum(1 for m in messages if m.__class__.__name__ == "AIMessage")
    tool_msgs = sum(1 for m in messages if m.__class__.__name__ == "ToolMessage")

    return f"Conversation has {human_msgs} user messages, {ai_msgs} AI responses, and {tool_msgs} tool results"
```

▼ Editing State Within Tool Function Demo

- Use `Command`

```
from langgraph.types import Command
from langchain.messages import RemoveMessage
from langgraph.graph.message import REMOVE_ALL_MESSAGES
from langchain.tools import tool, ToolRuntime

# Update the conversation history by removing all messages
@tool
def clear_conversation() → Command:
```

```

        """Clear the conversation history."""

    return Command(
        update={
            "messages": [RemoveMessage(id=REMOVE_ALL_MESSAGES)],
        }
    )

# Update the user_name in the agent state
@tool
def update_user_name(
    new_name: str,
    runtime: ToolRuntime
) → Command:
    """Update the user's name."""
    return Command(update={"user_name": new_name})

```

▼ Access Long Term Memory Store Within Tool Definitions Demo

```

from typing import Any
from langgraph.store.memory import InMemoryStore
from langchain.agents import create_agent
from langchain.tools import tool, ToolRuntime

# Access memory
@tool
def get_user_info(user_id: str, runtime: ToolRuntime) → str:
    """Look up user info."""
    store = runtime.store
    user_info = store.get(("users",), user_id)
    return str(user_info.value) if user_info else "Unknown user"

# Update memory

```

```

@tool
def save_user_info(user_id: str, user_info: dict[str, Any], runtime:
ToolRuntime) → str:
    """Save user info."""
    store = runtime.store
    store.put(("users",), user_id, user_info)
    return "Successfully saved user info."

store = InMemoryStore()
agent = create_agent(
    model,
    tools=[get_user_info, save_user_info],
    store=store
)

# First session: save user info
agent.invoke({
    "messages": [{"role": "user", "content": "Save the following use
r: userid: abc123, name: Foo, age: 25, email: foo@langchain.de
v"}]
})

# Second session: get user info
agent.invoke({
    "messages": [{"role": "user", "content": "Get user info for user
with id 'abc123'"}]
})
# Here is the user info for user with ID "abc123":
# - Name: Foo
# - Age: 25
# - Email: foo@langchain.dev

```



Some chat models (e.g., OpenAI, Anthropic, and Gemini) feature built-in tools that are executed server-side, such as web search and code interpreters. Refer to the provider overview to learn how to access these tools with your specific chat model. These tools are often called (AI message) and ran (Tool message) by the provider all in one.

Provide Tools to AI

Option 1: bind schema to a LLM model

- Best for individual model call
- Lets us easily attach tools schema to LLM system prompt
- If the model provider supports parallel tool calling then it can do that

```
from langchain.tools import tool

# Define tools
@tool
def get_weather(location: str) → str:
    """Get the weather at a location."""
    return f"It's sunny in {location}."

# Bind tools
model_with_tools = model.bind_tools([get_weather])

# View tool calls
ai_msg = model_with_tools.invoke("What's the weather like in Boston?")
for tool_call in ai_msg.tool_calls:
    # View tool calls made by the model
    print(f"Tool: {tool_call['name']}")
    print(f"Args: {tool_call['args']}")

# Execute tools and collect results
for tool_call in ai_msg.tool_calls:
    # Execute the tool with the generated arguments
    tool_result = get_weather.invoke(tool_call)
    messages.append(tool_result)

# Pass results back to model for next response
final_response = model_with_tools.invoke(messages)
print(final_response.text)
```

"The current weather in Boston is 72°F and sunny."

▼ Bind Tools & Force Model to Call Tool Demo

```
# Force model to call at least one of any of the tools
model_with_tools = model.bind_tools([tool_1], tool_choice="any")
```

```
# Force model to call a specific tool
model_with_tools = model.bind_tools([tool_1], tool_choice="tool_1")
```

Option 2: add tool node for ReACT agent

```
tools = [multiply]
tool_node = ToolNode(tools)
```

Structured Output

- Lets us just get structured output but not for tool calling

```
from pydantic import BaseModel, Field

class Movie(BaseModel):
    """A movie with details."""
    title: str = Field(..., description="The title of the movie")
    year: int = Field(..., description="The year the movie was released")
    director: str = Field(..., description="The director of the movie")
    rating: float = Field(..., description="The movie's rating out of 10")

model_with_structure = model.with_structured_output(Movie)
response = model_with_structure.invoke("Provide details about the movie Inception")
print(response.title)

Inception
```

Tool Error Handling

- `@wrap_tool_call` middleware lets us customise how tool errors are handled

```
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage

@wrap_tool_call
def handle_tool_errors(request, handler):
    """Handle tool execution errors with custom messages."""
    try:
        return handler(request)
    except Exception as e:
        # Return a custom error message to the model
        return ToolMessage(
            content=f"Tool error: Please check your input and try again. ({str(e)})",
            tool_call_id=request.tool_call["id"]
        )

agent = create_agent(
    model="gpt-4o",
    tools=[search, get_weather],
    middleware=[handle_tool_errors]
)
```

MCP Tools

▼ MCP Demo

- MCP is an asyn protocol so have to use it asynchronously

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent
```

```
client = MultiServerMCPClient(
    {
        "math": {
            "transport": "stdio", # Local subprocess communication
            "command": "python",
            # Absolute path to your math_server.py file
            "args": ["/path/to/math_server.py"],
        },
        "weather": {
            "transport": "streamable_http", # HTTP-based remote server
            # Ensure you start your weather server on port 8000
            "url": "http://localhost:8000/mcp",
        }
    }
)

tools = await client.get_tools()
agent = create_agent(
    "claude-sonnet-4-5-20250929",
    tools
)
math_response = await agent.ainvoke(
    {"messages": [{"role": "user", "content": "what's (3 + 5) x 12?"}]}
)
```

```
weather_response = await agent.ainvoke(  
    {"messages": [{"role": "user", "content": "what is the weather i  
n nyc?"}]}  
)
```

▼ MCP Stateful Demo

- By default MCP is stateless meaning context isn't persisted in tool functions.
We can change this by using a client session

```
from langchain_mcp_adapters.tools import load_mcp_tools  
  
client = MultiServerMCPClient({...})  
async with client.session("math") as session:  
    tools = await load_mcp_tools(session)
```